```
    return []

else
     #3
    points ←
[]
   #4
     if
rectangle[i].min≤node.point[i]≤rectangle[i].max "
0≤i<k then    #5
       points.insert(node.point)
     if intersectLeft(rectangle, node)
then                                #6
       points.insertAll(pointsInRectangle(node.left,
rectangle))
    if intersectRight(rectangle, node)
then                               #7
      points.insertAll(pointsInRectangle(node.right,
rectangle))
    return
points
    #8
```

#1 Find all the points in the container intersecting a given hyper-sphere. We pass the hyper-rectangle as an argument, we can assume it is a list of named tuples, each containing the boundaries for a dimension of the rectangle, as a range (min-max values).

#2 If node is `null`, we are traversing an empty tree, so there is no point to be added.

#3 Otherwise, we have 3 tasks: check if current node is inside the hyper-rectangle, traverse the branch (at least one) or branches that intersects with the rectangle